

PARTE TEORIA (12 PUNTI)

Domanda 1

Punteggio max.:
2,00

Contrassegna
domanda

Sia data la sequenza di interi, supposta memorizzata in un vettore:

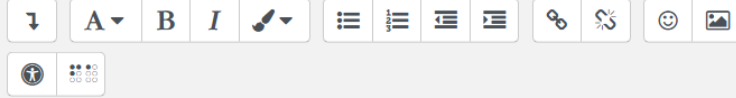
27 22 26 51 23 87 31 38 28 61 20 25 29 30

la si trasformi in un heap, ipotizzando di usare un vettore come struttura dati.

Si riportino graficamente i passi significativi della costruzione dell'heap ed il risultato finale.

Si ipotizzi che, alla fine, nella radice dell'heap sia memorizzato il valore minimo si eseguano su tale heap i primi 2 passi dell'algoritmo di heapsort.

NB: la sequenza è già memorizzata nel vettore e rappresenta una configurazione intermedia per cui la proprietà di heap non è ancora soddisfatta.



Formato della risposta:

su una sola riga elenco elementi separati da virgole o spazi:

el1, el2, el3, ... eln

Si elenchi il contenuto del vettore dell'heap al termine dell'algoritmo di trasformazione.

vettore al termine dell'algoritmo di trasformazione:

Si elenchi il contenuto del vettore dell'heap dopo l'esecuzione dei primi 2 passi di heapsort.

vettore dell'heap dopo il primo passo di heapsort:

vettore dell'heap dopo il secondo passo di heapsort:

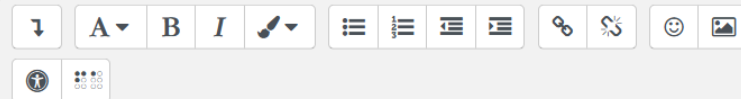
Domanda 2

Punteggio max.:
1,50

Contrassegna
domanda

Si supponga di aver memorizzato numeri tra 0 e 400 in un BST e di cercare la chiave 255. Supponendo il BST corretto, è possibile incontrare le seguenti sequenze durante la ricerca? Motivare brevemente la risposta.

- 2 252 309 220 330 264 255
- 2 252 209 220 240 250 255
- 2 400 202 300 270 250 255



La sequenza 2 252 309 220 330 264 255 può essere incontrata?

Sì/No

Se non può essere incontrata, indicare la prima violazione incontrata e spiegarla

La sequenza 2 252 209 220 240 250 255 può essere incontrata?

Sì/No

Se non può essere incontrata, indicare la prima violazione incontrata e spiegarla

La sequenza 2 400 202 300 270 250 255 può essere incontrata?

Sì/No

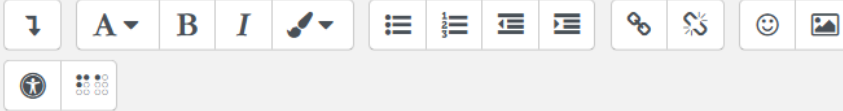
Se non può essere incontrata, indicare la prima violazione incontrata e spiegarla

Domanda 3

Punteggio max.:
2,00

Contrassegna
domanda

Sia data la sequenza di chiavi intere 211, 26, 79, 46, 154, 17, 43, 230, 16. Si riporti il contenuto di una tabella di hash, inizialmente supposta vuota, in cui avvenga l'inserimento della sequenza indicata. Si usi l'open addressing con linear probing. Si determini la dimensione minima M della tabella di per avere un fattore di carico $\alpha < 0.4$.



Formato della risposta:

dimensione minima della tabella

$M =$

Quale tra le seguenti funzioni di hash è opportuno usare?

$h_1(k) = k \% 17, h_2(k) = 1 + k \% 17$

$h(k) = k \% 23$

$h(k) = k \% 19$

$h_1(k) = k \% 23, h_2(k) = 1 + k \% 97$

Si motivi brevemente la risposta.

Collisioni chiave 16:

cella1 dove vi è collisione:

...

cellaN dove vi è collisione:

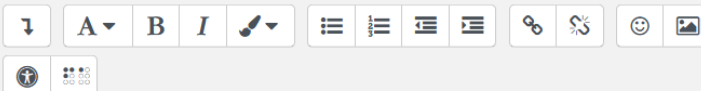
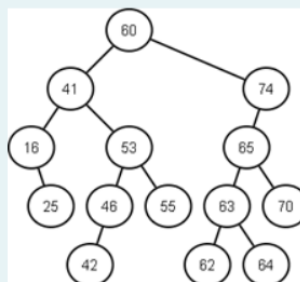
Al termine dell'inserzione, in quale cella si trova la chiave 230?

Domanda 4

Punteggio max.:
2,00

Contrassegna
domanda

Si inserisca in **radice** nel BST di figura la chiave 45 e poi si cancelli la chiave 60.



Dopo l'inserimento di 45, chi sono i figli sinistro e destro della radice?

figlio SX=

figlio DX =

Dopo la cancellazione di 60, chi sono i figli sinistro e destro di 65?

figlio SX=

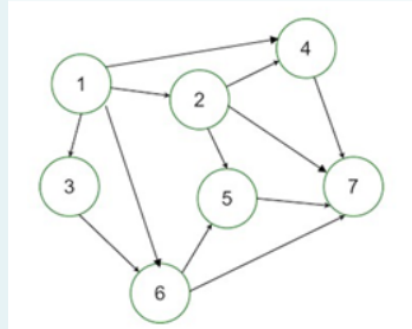
figlio DX =

Domanda 5

Punteggio max.:
2,00

Contrassegna
domanda

Si ordini topologicamente il seguente DAG. Si consideri **1** come vertice di partenza. Qualora necessario, si trattino i vertici secondo l'ordine numerico e si assuma che la lista delle adiacenze sia anch'essa ordinata numericamente.



Richiedi aiuto

Formattazione: **B** *I*

Formato della risposta:

per ogni vertice tempo di scoperta / tempo di fine elaborazione

Vertice 1:

Vertice 2:

Vertice 3:

Vertice 4:

Vertice 5:

Vertice 6:

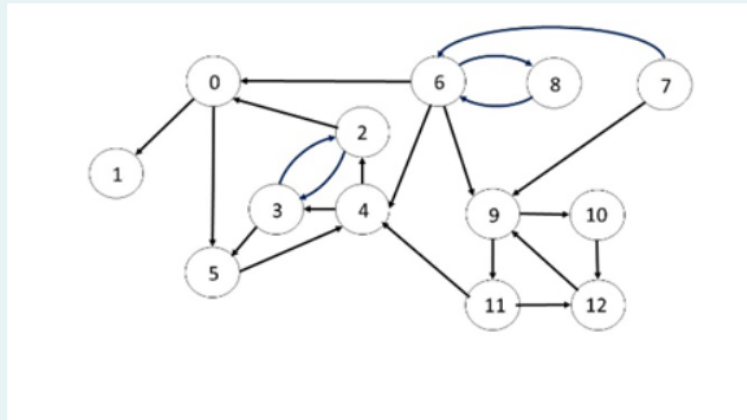
Vertice 7:

Domanda 6

Punteggio max.:
2,50

Contrassegna
domanda

Dato il seguente grafo orientato:



Si applichi l'algoritmo di Kosaraju per il calcolo delle componenti fortemente connesse, considerando **0** come vertice di partenza. Qualora necessario, si trattino i vertici secondo l'ordine numerico e si assuma che la lista delle adiacenze sia anch'essa ordinata numericamente.

Richiede un account per rispondere a questa domanda.

Formato della risposta:

Elencazione delle SCC come insiemi di vertici:

SCC1 =

SCC2 =

....

SCCn =

Domanda 1

Punteggio max.: 2,00

Contrassegna domanda

Sia data la sequenza di interi, supposta memorizzata in un vettore:

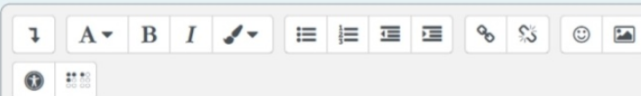
27 22 26 51 23 87 31 38 28 61 20 25 29 30

la si trasformi in un heap, ipotizzando di usare un vettore come struttura dati.

Si riporti graficamente i passi significativi della costruzione dell'heap ed il risultato finale.

Si ipotizzi che, alla fine, nella radice dell'heap sia memorizzato il valore minimo si eseguano su tale heap i primi 2 passi dell'algoritmo di heapsort.

NB: la sequenza è già memorizzata nel vettore e rappresenta una configurazione intermedia per cui la proprietà di heap non è ancora soddisfatta.



Formato della risposta:

su una sola riga elenco elementi separati da virgole o spazi:

el1, el2, el3, ... eln

Si elenchi il contenuto del vettore dell'heap al termine dell'algoritmo di trasformazione.

vettore al termine dell'algoritmo di trasformazione:

Si elenchi il contenuto del vettore dell'heap dopo l'esecuzione dei primi 2 passi di heapsort.

vettore dell'heap dopo il primo passo di heapsort:

vettore dell'heap dopo il secondo passo di heapsort:

Trasformazione in heap

1. Le foglie sono già heap. Si parte dai padri delle foglie.
2. Si confronta la radice con il figlio sinistro.
 - a. Se l'elemento è minore del figlio sinistro, allora si propone il figlio sinistro come nuova radice del sotto-albero.
 - b. Altrimenti si propone l'elemento come radice del sotto-albero.
3. Si confronta l'elemento candidato con il figlio destro.

di teoria

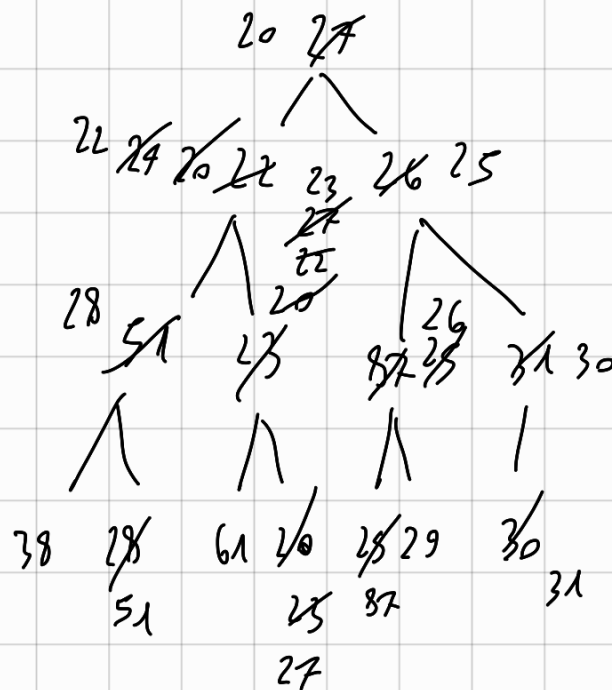
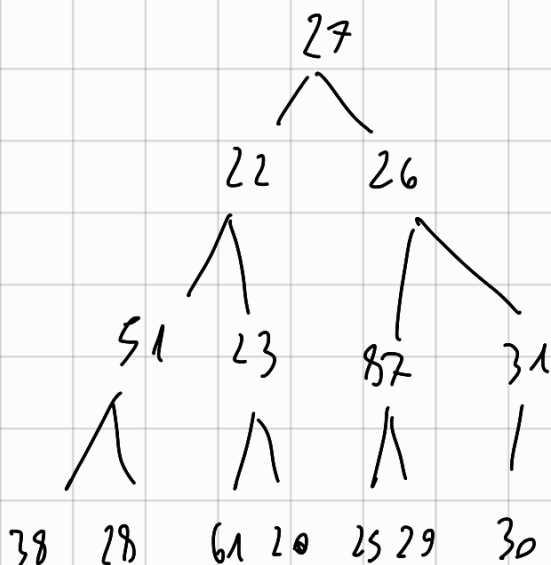
- a. Se l'elemento candidato è minore del figlio destro, allora si propone il figlio destro come nuova radice del sotto-albero.
 - b. Altrimenti si propone l'elemento come radice del sotto-albero.
4. L'elemento candidato diventa la radice del sotto-albero, sostituendo la radice attuale.
 5. Si ripete con tutti i nodi dal basso verso l'alto.

Heapsort

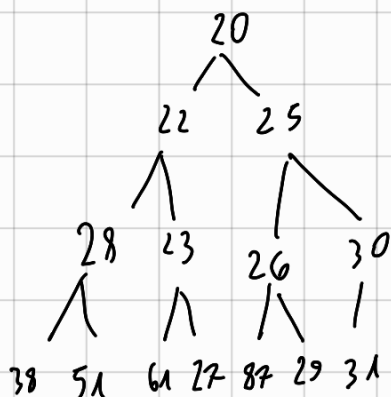
1. Si scambia l'elemento alla radice dell'albero con l'ultimo elemento presente nell'heap, e lo si salva in un vettore alla posizione i , dove i indica il numero del passo da 0 a n .
2. Si applica la trasformazione in heap, mantenendo l'elemento alla fine dell'heap appena scambiato nella stessa posizione.
3. Applica questo procedimento per ogni elemento fino ad ottenere l'ordinamento.

27 22 26 51 23 87 31 38 28 61 20 25 29 30

Trasformazione: Heapify

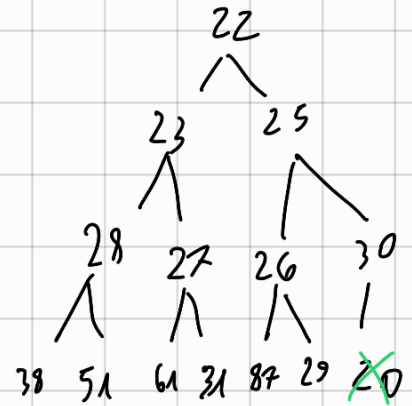
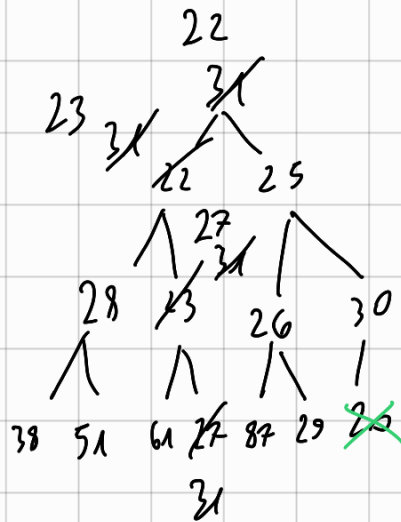
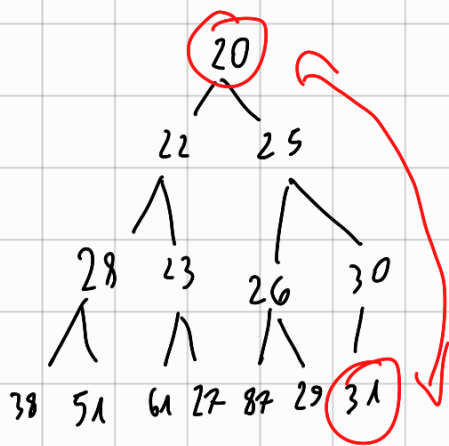


Trasformazione: Heapify



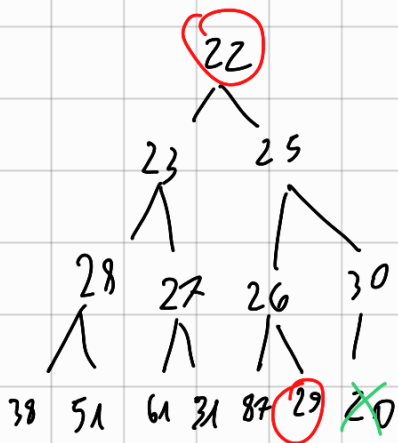
→ [20, 22, 25, 28, 23, 26, 30, 38, 51, 61, 26, 87, 29, 31]

• H: AP 50R5 1

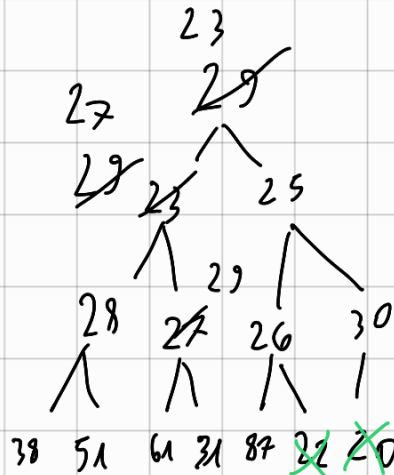


1º Passo [22, 23, 25, 28, 27, 26, 30, 38, 51, 61, 31, 87, 29]
(20 Fuori)

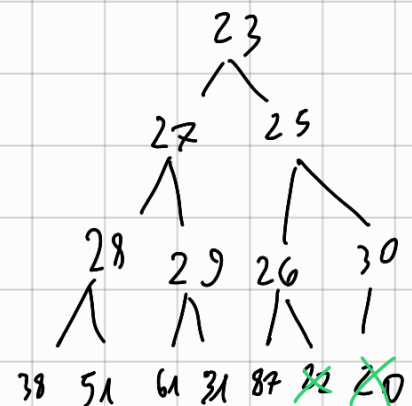
• H: AP 50R5 2



→



→



2º Passo [23, 27, 25, 28, 29, 26, 30, 38, 51, 61, 31, 87]
(22, 20 Fuori)

Domanda 2

Punteggio max:
1,50

Contrassegna
domanda

Si supponga di aver memorizzato numeri tra 0 e 400 in un BST e di cercare la chiave 255.
Supponendo il BST corretto, è possibile incontrare le seguenti sequenze durante la ricerca?
Motivare brevemente la risposta.

- 2 252 309 220 330 264 255
- 2 252 209 220 240 250 255
- 2 400 202 300 270 250 255

La sequenza 2 252 309 220 330 264 255 può essere incontrata?

Si/No

Se non può essere incontrata, indicare la prima violazione incontrata e spiegarla

La sequenza 2 252 209 220 240 250 255 può essere incontrata?

Si/No

Se non può essere incontrata, indicare la prima violazione incontrata e spiegarla

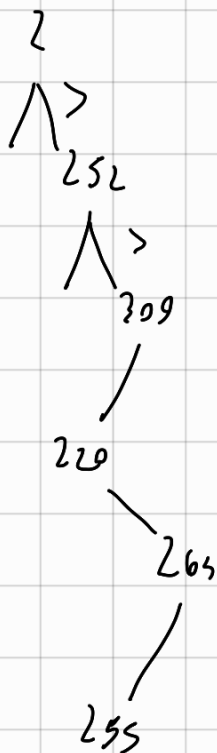
La sequenza 2 400 202 300 270 250 255 può essere incontrata?

Si/No

Se non può essere incontrata, indicare la prima violazione incontrata e spiegarla

Lisotico

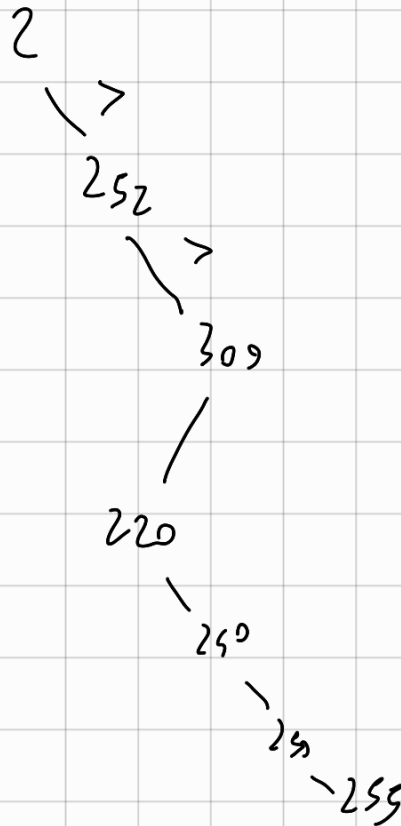
1°



NO

220 < 252
NON PUÒ ESSERE
A DESTRA

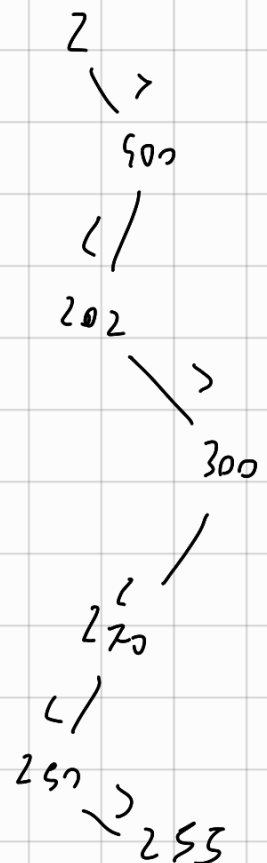
2°



NO

CONCILI PRIMA

3°



SI

BST USUETTATO

Domanda 3

Punteggio max:
2,00

Contrassegna
domanda

Sia data la sequenza di chiavi intere 211, 26, 79, 46, 154, 17, 43, 230, 16. Si riporti il contenuto di una tabella di hash, inizialmente supposta vuota, in cui avvenga l'inserimento della sequenza indicata. Si usi l'open addressing con linear probing. Si determini la dimensione minima M della tabella di per avere un fattore di carico $\alpha < 0.4$.

Formato della risposta:

dimensione minima della tabella

$M = 23$

Quale tra le seguenti funzioni di hash è opportuno usare?

$h_1(k) = k \% 17, h_2(k) = 1+k \% 17$ ☒

$h(k) = k \% 23$ ☒

$h(k) = k \% 19$ ☒

$h_1(k) = k \% 23, h_2(k) = 1+k \% 97$ ☒

Si motivi brevemente la risposta.

Collisioni chiave 16:

cella1 dove vi è collisione: 16

...

cellaN dove vi è collisione: 17

Al termine dell'inserzione, in quale cella si trova la chiave 230? 1

$$N: 9$$

$$\alpha = \frac{N}{M} < 0.4$$

$$9$$

$$\frac{9}{M} < 0.4$$

$$M > \frac{9}{0.4} = 22.5$$

$$h(N): (N+i) \bmod 23 \approx 23 \text{ (non prima)}$$

non ho voglia di digitare

$$\cdot 211 \bmod 23; 4$$

$$\cdot 26; 3$$

$$\cdot 79; 10$$

$$\cdot 46; 0$$

$$\cdot 154; 16$$

$$\cdot 17; 17$$

$$\cdot 43; 20$$

$$\cdot 230; 0 \rightarrow 1$$

$$\cdot 16; 16 \rightarrow 17 \rightarrow 18$$

Linear Probing

1. Definiamo la funzione di Hashing.

$$h(k) = (k + i) \% M$$

2. Si applica la chiave di Hashing al valore k definito dalla funzione di Hashing con inizialmente $i = 0$.

a. Se la cella con indice $h(k)$ è vuota, allora inserisci il valore nella cella.

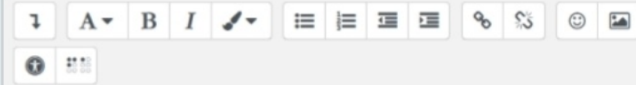
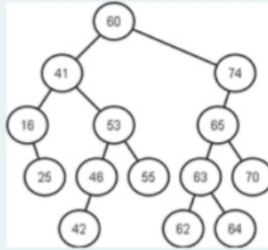
b. Se non è vuota, allora tenta l'inserimento con $i = i + 1$, e si ripete fino al successo.

Domanda 4

Punteggio max.:
2,00

Contrassegna
domanda

Si inserisca in **radice** nel BST di figura la chiave 45 e poi si cancelli la chiave 60.



Dopo l'inserimento di 45, chi sono i figli sinistro e destro della radice?

figlio SX=

figlio DX =

Dopo la cancellazione di 60, chi sono i figli sinistro e destro di 65?

figlio SX=

figlio DX =

Inserimento in radice

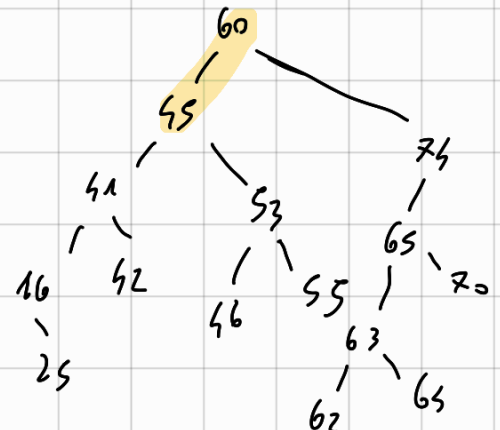
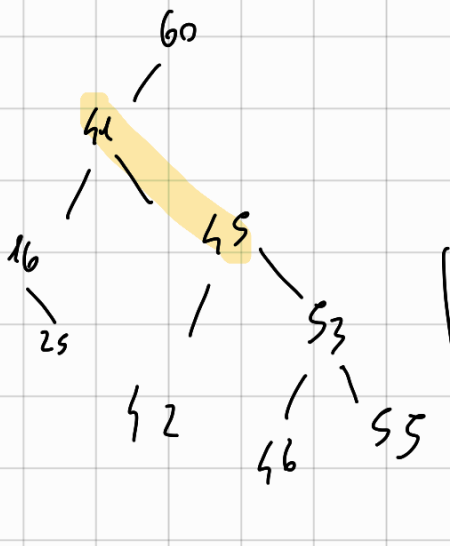
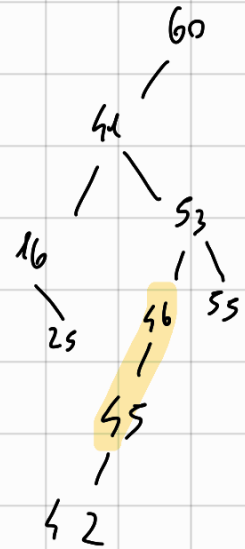
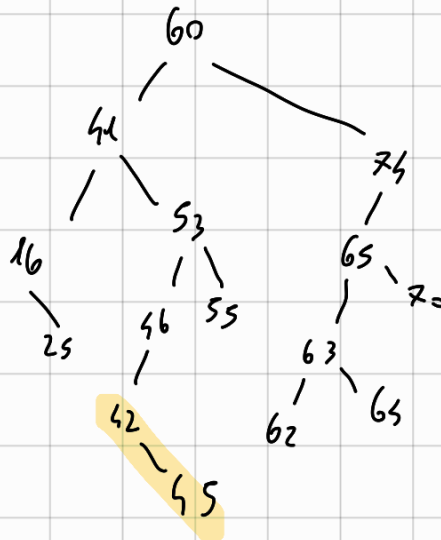
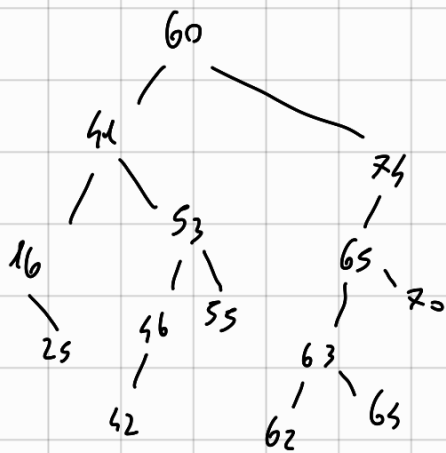
1. Se l'albero è vuoto, si inserisce l'elemento da inserire nella radice.
2. Se non è vuoto, si inserisce nell'albero inserendolo in foglia.
3. Si confronta la foglia appena inserita con quella padre, allo scopo di mantenere le proprietà del BST.

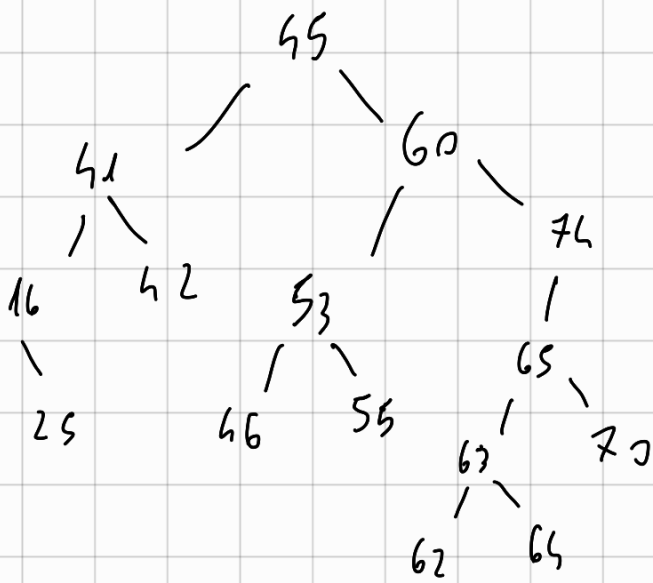
- a. Se l'elemento non è in posizione ed è il figlio destro, allora si ruota verso sinistra.
 - i. Il padre del figlio destro diventa il padre del padre.
 - ii. Il figlio sinistro del figlio destro diventa il figlio destro del padre.
- b. Se l'elemento non è in posizione ed è il figlio sinistro, allora si ruota verso destra.
 - i. Il padre del figlio sinistro diventa il padre del padre.
 - ii. Il figlio destro del figlio sinistro diventa il figlio sinistro del padre.
- c. Se l'elemento è in posizione non si fa nulla.

Cancellazione

1. Si ricerca la posizione della chiave e si rimuove l'elemento.
2. Se l'elemento eliminato è una radice, si effettua un partizionamento della radice dell'albero destro.
3. L'elemento sinistro della radice diventa l'albero sinistro ottenuto dalla rimozione dell'elemento.

INS 45 IN Radice



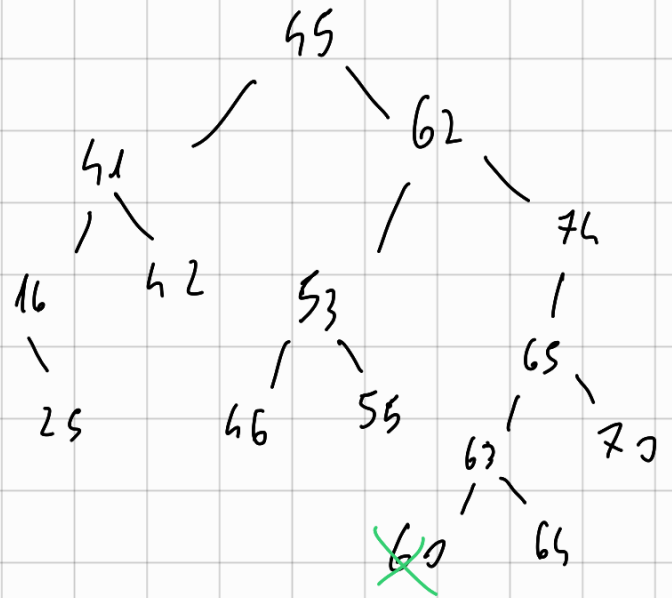
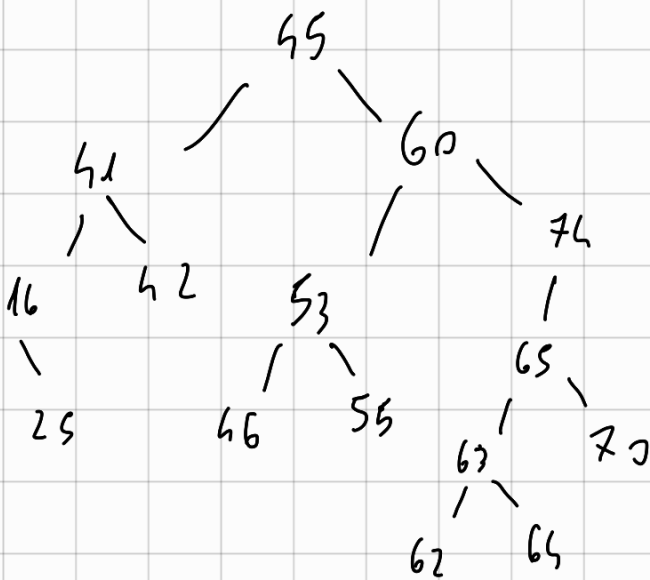


• CANCELLA 60

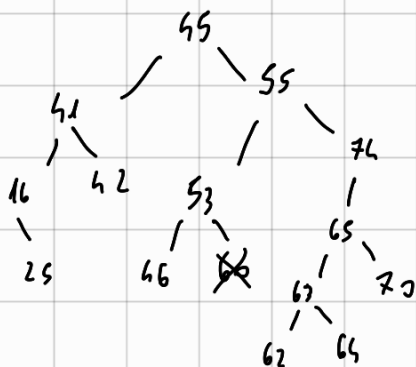
USO SUCCESSO

(102 62)

(LILLO MINIMO A DESTRA)



SE USATO MODULAZIONE: (MAX SIMILITUDINE) 55



USATO STA

NON

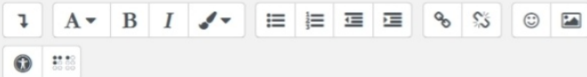
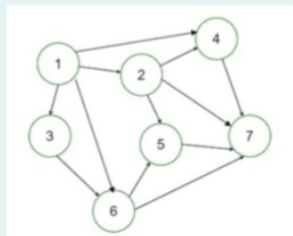
CAMBIA

Domanda 5

Punteggio max.:
2,00

Contrassegna
domanda

Si ordini topologicamente il seguente DAG. Si consideri **1** come vertice di partenza. Qualora necessario, si trattino i vertici secondo l'ordine numerico e si assuma che la lista delle adiacenze sia anch'essa ordinata numericamente.



Formato della risposta:

per ogni vertice tempo di scoperta / tempo di fine elaborazione

Vertice 1:

Vertice 2:

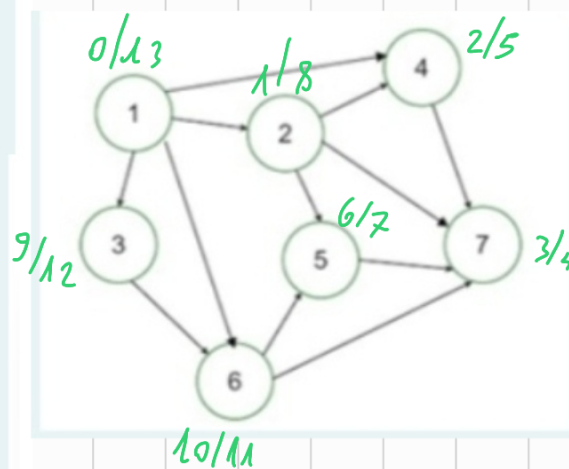
Vertice 3:

Vertice 4:

Vertice 5:

Vertice 6:

Vertice 7:



Ordinamento topologico del DAG

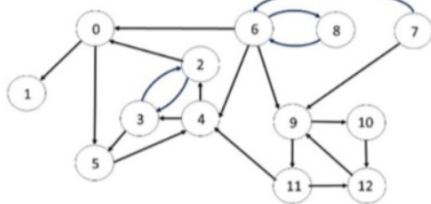
1. Si effettua una visita in profondità.
2. Si ordinano i nodi per ordine inverso di tempo di terminazione di ogni nodo.

Domanda 6

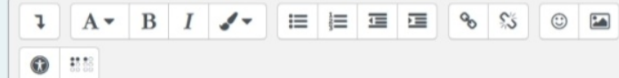
Punteggio max:
2,50

Contrassegna domanda

Dato il seguente grafo orientato:



Si applichi l'algoritmo di Kosaraju per il calcolo delle componenti fortemente connesse, considerando 0 come vertice di partenza. Qualora necessario, si trattino i vertici secondo l'ordine numerico e si assuma che la lista delle adiacenze sia anch'essa ordinata numericamente.



Formato della risposta:

Elencazione delle SCC come insiemi di vertici:

SCC1 =

SCC2 =

....

SCCn =

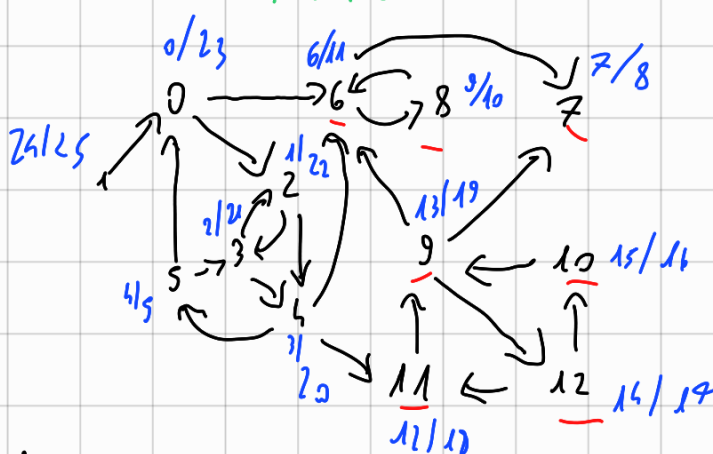
Algoritmo di Kosaraju

1. Si effettua una visita in profondità su un grafo trasposto.
 - a. Il grafo trasposto ha gli archi inversi rispetto al grafo originale.
2. Si effettua la stessa visita a partire dall'ordine inverso dei tempi di terminazione di ogni nodo.
 - a. Ogni volta che viene effettuata una visita in profondità, si inizia una componente fortemente connessa.
3. Tutte le volte che viene iniziata la visita in profondità da un nodo è definita una componente fortemente connessa.

ORIGINALI



TRASPOSTO



1, 0, 2, 3, 4, 5, 11, 12, 10, 6, 8, 7, 9

SCC1 = 1
SCC2 = 0, 5, 4, 3, 2
SCC3 = 9, 12, 10, 11
SCC4 = 6, 8,
SCC5 = 7